PROVISIONAL PATENT APPLICATION

SYSTEM AND METHOD FOR AUTOMATED SCHEMA DEDUPLICATION AND CANONICAL FIELD RESOLUTION USING SEMANTIC SIMILARITY WITH INFORMATION-THEORETIC TRANSFORM CLASSIFICATION

Authors: Tom Tang, Fei Jia

Organization: Edge Vector Foundation

Date: March 30, 2026

Applicant

Edge Vector Foundation
2328 Delmar Dr E, Seattle, WA 98102

Inventors

- Tom Tang, Seattle, WA
- Fei Jia, Kenmore, WA

Field of the Invention

This invention relates to schema management in distributed database systems, and more particularly to a system and method for automatically detecting semantically equivalent schemas and fields across independent data sources, resolving fields to canonical names, and classifying the sensitivity of data transformations using information-theoretic analysis.

Cross-Reference to Related Applications

None (provisional filing).

Background of the Invention

In distributed systems where multiple independent nodes ingest data from diverse sources, the same real-world concept is frequently represented by different schema names, field names, and data structures. For example, one node may ingest recipe data with a field called "ingredients" while another uses "ingredient_list" — both representing the same semantic concept. Without a mechanism to detect and reconcile these equivalences, the system accumulates duplicate schemas that fragment what should be unified data.

Existing approaches to schema matching rely on syntactic comparison (exact name matching, edit distance) or require manual mapping by database administrators. These approaches fail when:

- Different data sources use different naming conventions for the same concept
- The same field name is used with different semantic meanings across domains
- New schemas arrive continuously from automated ingestion pipelines without human review
- The system operates at scale across thousands of independent nodes

Additionally, when transforms (computed functions) operate on classified data fields, determining the sensitivity classification of the transform's output is a non-trivial problem. A transform that reads a highly sensitive medical diagnosis field but outputs only a boolean "has_condition" flag should receive a lower classification than one that passes the diagnosis through unchanged. Existing approaches either conservatively assign the maximum input classification to all outputs (overly restrictive) or require manual classification by security analysts (does not scale).

There is a need for an automated system that resolves semantically equivalent schemas and fields to canonical representations, prevents duplication across distributed nodes, and automatically classifies the sensitivity of data transforms using rigorous information-theoretic methods.

Summary of the Invention

The present invention provides a system and method comprising:

1. **A schema identity hashing system** that computes a deterministic identity for each schema based on its human-readable descriptive name and sorted, deduplicated field names, using SHA-256, such that semantically identical schemas (same readable name + same fields) produce the same identity hash regardless of originating node, and schemas differing in either name or fields produce different hashes.
2. **A semantic similarity detection system** using neural embedding models that:
   - Embeds schema descriptive names and detects name-level similarity above a configurable threshold (default 0.8 cosine similarity)
   - Embeds field descriptions (excluding field names to avoid noise from naming conventions) and detects field-level similarity above a separate threshold (default 0.88)

- Applies bidirectional best-match verification to prevent many-to-one field mappings: a candidate canonical match is accepted only if the canonical field's best match is also the incoming field

3. **A global canonical field registry** that maintains a single authoritative mapping of canonical field names to their descriptions, types, data classifications, and interest categories, where:
   - New fields are matched against existing canonical fields using embedding similarity with bidirectional verification
   - Matched fields are renamed to the canonical name with mutation mappers tracking the rename for data transformation
   - Unmatched fields are registered as new canonical entries
   - Field descriptions are built using AI-generated descriptions when available, falling back to classification-based descriptions with schema context suffixes
4. **A schema expansion mechanism** where:
   - A new schema whose descriptive name matches an existing schema but contains additional fields triggers expansion rather than duplication
   - The existing schema is blocked (deprecated)
   - A new superset schema is created with field mappers pointing shared fields to the existing schema's molecules (data storage units) and fresh molecules for new fields
   - The expansion uses semantic field rename matching (cosine similarity threshold 0.84) with context-enriched embeddings to align fields between old and new schemas
5. **An information-theoretic transform classification system** that automatically determines the data sensitivity of transform outputs using:
   - **Phase 1 (Input Ceiling):** The transform's input queries are resolved against the schema service to determine the maximum classification of all input fields, establishing an upper bound
   - **Phase 2 (Output Floor via Mutual Information):** The transform WASM module is executed against synthetic test data (512–1024 samples per input field), and Normalized Mutual Information (NMI) is measured between each input-output field pair to determine how much information about each sensitive input is preserved in each output field
   - Fields with NMI above a leakage threshold (default 0.1) inherit the classification of the correlated input field
   - Fields with NMI below the threshold are classified independently (confirmed information reduction)
   - The overall transform classification is the maximum across all output field classifications

Detailed Description of the Invention

System Architecture

The system comprises a centralized schema service that maintains the canonical registry, and distributed client nodes that submit schemas for registration and receive canonical mappings. The schema service operates as a standalone HTTP server with persistent storage (an embedded key-value store for local deployment, a managed cloud database for cloud deployment).

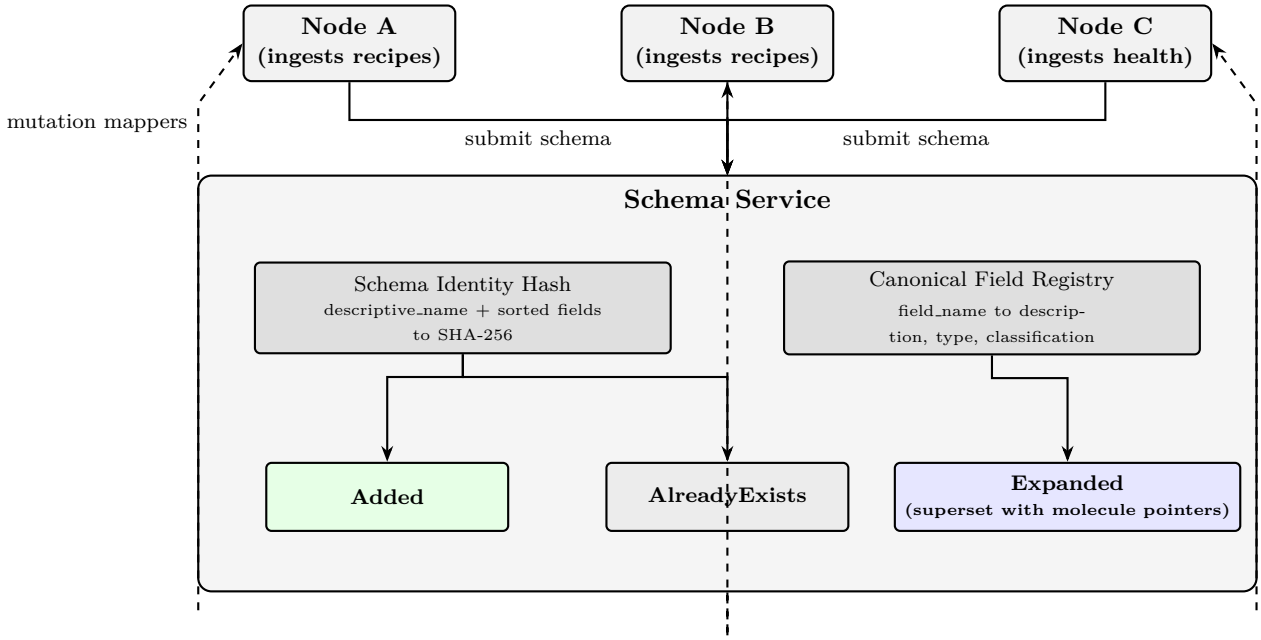

Figure 1: System architecture. Distributed nodes submit schemas to a centralized schema service. The service computes identity hashes and resolves fields to canonical names, returning one of three outcomes (Added, AlreadyExists, Expanded) along with mutation mappers for data transformation.

Schema Identity Hash

Each schema's identity is computed deterministically:

1. Determine the readable name: prefer the descriptive name (human-readable label) if present; otherwise use the schema's programmatic name
2. Collect all field names, sort alphabetically, and remove duplicates
3. Concatenate as: `"{readable_name}:{field1},{field2},...,{fieldN}"`
4. Compute SHA-256 hash of the concatenated string

The schema is stored using the identity hash as its canonical name. This ensures:

- **Idempotent registration:** Submitting the same schema twice returns the existing entry rather than creating a duplicate
- **Cross-node deduplication:** Two independent nodes ingesting semantically identical data produce the same identity hash and are resolved to the same canonical schema
- **Deterministic identity:** The hash depends only on the semantic content (readable name + fields), not on ordering, formatting, or metadata

Schema Addition Outcomes

When a schema is submitted for registration, three outcomes are possible:

1. **Added:** No existing schema matches the identity hash or descriptive name. The schema is registered as new.
2. **AlreadyExists:** An existing schema has the same identity hash. The existing schema is returned with its canonical field mappings.
3. **Expanded:** An existing schema has the same descriptive name but different fields (the new schema is a superset). The old schema is blocked, a new superset schema is created with field mappers preserving data continuity, and the descriptive name index is updated to point to the new schema.

Semantic Descriptive Name Matching

The system maintains a cache of embedding vectors for all registered schema descriptive names. When a new schema arrives:

1. Compute the embedding vector for the new schema's descriptive name using a neural embedding model (e.g., all-MiniLM-L6-v2, 384 dimensions)
2. Compute cosine similarity against all cached descriptive name embeddings
3. If the maximum similarity exceeds the descriptive name threshold (default 0.8), identify the matching schema
4. If the similarity is below threshold, treat as a new schema

This detects semantic equivalence even when naming conventions differ (e.g., "Recipe Collection" vs. "Cooking Recipes" vs. "My Recipes").

Canonical Field Resolution

The canonical field registry maps field names to their authoritative definitions:

```
canonical_field_name -> {
    description: String,
    field_type: FieldValueType,
    classification: DataClassification,
    interest_category: Option<String>
}
```

When a new schema's fields are canonicalized:

1. For each incoming field not already in the canonical registry:
   a. Build a description string for the field, preferring AI-generated descriptions over fallback heuristics
   b. Compute the embedding vector of the description (excluding the field name to avoid bias from naming conventions)
   c. Find the canonical field with the highest cosine similarity above the field similarity threshold (default 0.88)
   d. Apply bidirectional verification: confirm that the canonical field's best match among all incoming fields is also this field (prevents many-to-one mappings)
   e. If verified: rename the incoming field to the canonical name and record a mutation mapper for data transformation
   f. If no match or verification fails: register the field as a new canonical entry
2. For matched fields, the mutation mapper records the rename: {`original_field_name` → `canonical_field_name`}. This mapping is returned to the submitting node so that data records can be transformed to use canonical field names.

Bidirectional Verification. The bidirectional check prevents degenerate mappings.

Without it, if canonical field "description" is the closest match for both incoming "notes" and incoming "summary," both would be mapped to "description," losing distinction. With bidirectional verification:

- "notes" matches "description" with similarity 0.90
- "summary" matches "description" with similarity 0.89
- From "description"'s perspective, best match is "notes" (0.90)
- Therefore only "notes" maps to "description"; "summary" becomes a new canonical field

Embedding Strategy. Field descriptions are embedded rather than field names because different data sources use different naming conventions for the same concept. "ingredients," "ingredient_list," "recipe_items," and "components" may all refer to the same concept. By embedding the semantic description ("the list of ingredients used in a recipe"), the system captures meaning rather than syntax.

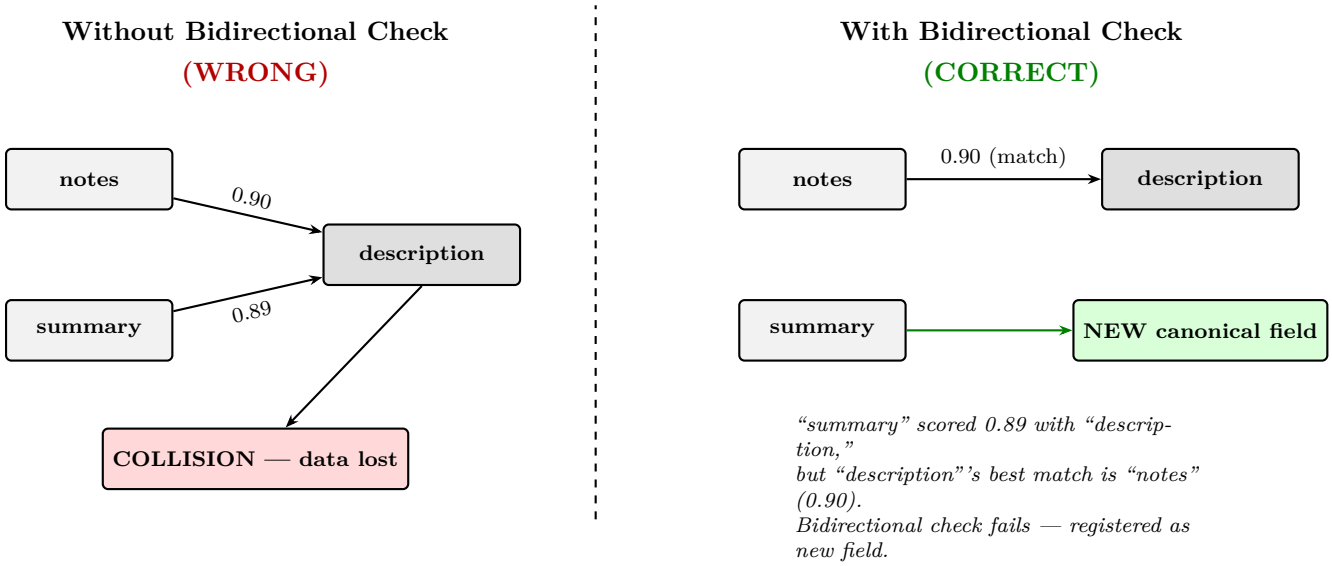

Figure 2: Bidirectional verification prevents many-to-one mappings. Without the check (left), both "notes" and "summary" map to "description," causing data loss. With the check (right), only "notes" maps to "description" because it is the best match in both directions; "summary" becomes a new canonical field.

Schema Expansion

When a new schema shares its descriptive name with an existing schema but contains additional fields:

1. Detect the overlap: the new schema matches an existing schema's descriptive name but its identity hash differs (different fields)
2. Compute a semantic field rename map between old and new schema fields using context-enriched embeddings with format "the {name} of the {context}: {description}" at a lower similarity threshold (default 0.84)
3. Block (deprecate) the old schema
4. Create the new superset schema:
   - For fields that exist in both old and new: create field mappers pointing to the old schema's molecules (data storage units), preserving existing data without migration
   - For new fields: allocate fresh molecules
5. Apply field mappers: copy molecule UUIDs from the source schema to the expanded schema, ensuring data continuity
6. Update the descriptive name index to point to the new schema's identity hash

This enables non-destructive schema evolution: adding fields to a concept creates a superset schema that inherits all existing data through molecule-level pointers, with no data migration required.

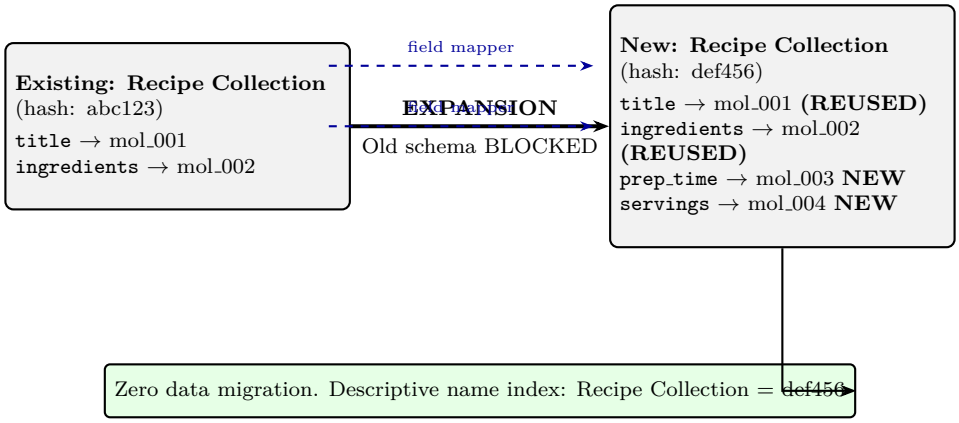

Figure 3: Schema expansion. When a new schema shares its descriptive name with an existing schema but contains additional fields, the old schema is blocked and a new superset schema is created. Shared fields reuse molecule pointers via field mappers; new fields receive fresh molecules. No data migration is required.

Collection Name Validation

To prevent semantically meaningless schema names from entering the registry, the system validates descriptive names against a set of pre-computed anchor embeddings representing common collection concepts (e.g., "Photo Collection," "Recipe Collection," "Medical Records," "Financial Transactions"). Names with cosine similarity below a minimum threshold (default 0.3) to all anchors are flagged as potentially invalid.

Data Classification

Each canonical field receives a data classification comprising sensitivity level and data domain:

1. If the submitting node provides a classification, it is used directly
2. If no classification is provided, the schema service uses a large language model to infer the classification from the field description
3. Classification is never omitted — the service returns an error rather than registering an unclassified field

Classification is the schema service's responsibility, not the data ingestion pipeline's. This ensures consistent, centralized classification authority across all nodes and data sources.

Interest Category Inference

Each canonical field optionally receives an interest category (e.g., "Photography," "Cooking," "Health & Fitness") inferred by a language model from the field description. Interest categories are best-effort — inference failures do not block schema registration. Categories are used for discovery and personalization features downstream.

Information-Theoretic Transform Classification

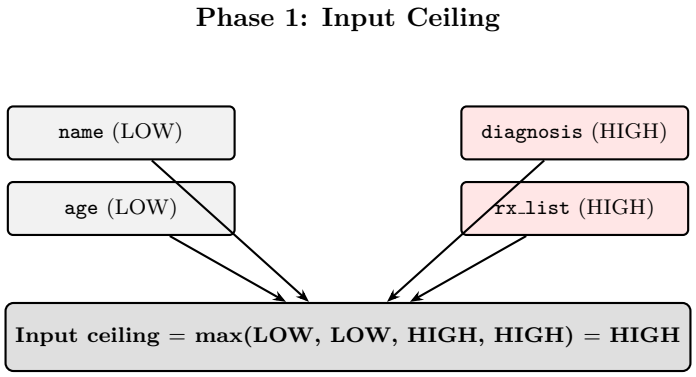

Figure 4: Phase 1: Input Ceiling. The maximum classification across all input fields establishes an upper bound for the transform's output classification.

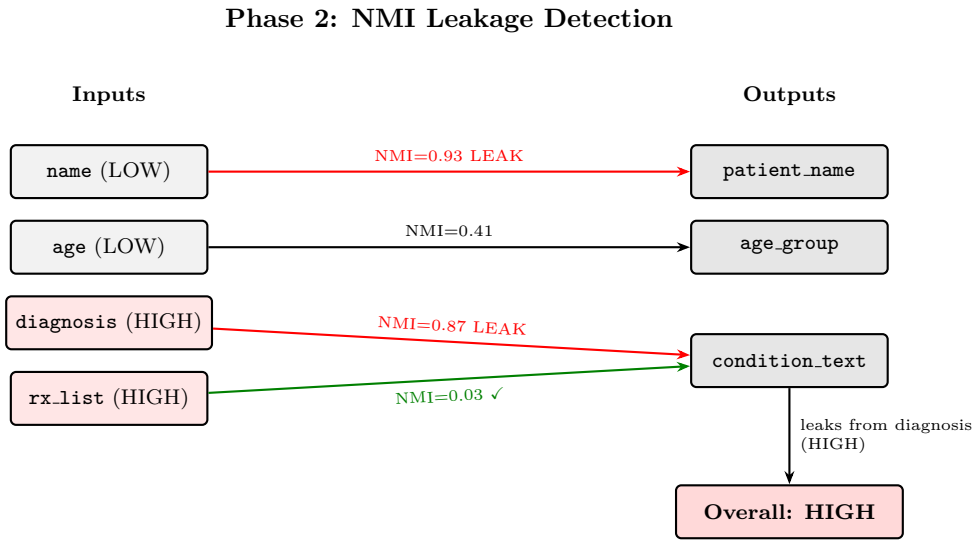

Figure 5: Phase 2: NMI Leakage Detection. For each input-output field pair, Normalized Mutual Information is computed over synthetic samples. Fields with NMI above the leakage threshold (0.1) inherit the classification of the correlated input field. The overall transform classification is the maximum across all output field classifications.

When WASM transforms are registered with the schema service, the system automatically determines the data sensitivity of the transform's output using a two-phase information-theoretic analysis.

Phase 1: Input Ceiling

The transform declares its inputs as queries against registered schemas. The schema service resolves the data classification of each input field from the canonical registry. The input ceiling is the maximum classification across all input fields.

This establishes an upper bound: a transform reading HIGH-classified fields cannot produce output classified higher than HIGH.

Phase 2: Output Floor via Normalized Mutual Information

To determine whether the transform genuinely reduces information (e.g., aggregating detailed records into summary statistics), the service runs the WASM module against synthetic test data:

1. **Synthetic Data Generation:** For each input field, generate N samples (default 512–1024) drawn from type-appropriate distributions:
   - String (name/ID): random alphanumeric strings
   - String (text): random sentences from fixed vocabulary
   - Integer: uniform distribution over plausible range
   - Float: Gaussian with plausible mean and standard deviation
   - Boolean: Bernoulli(0.5)
   - Array: length 1–5, each element from the scalar distribution
   - Enum: uniform over known values
2. **Isolated Field Variation:** For each sensitive input field X_i, vary X_i across N samples while holding all other input fields at fixed baseline values. Run the WASM transform on each sample. Collect N output values for each output field Y_j.
3. **Normalized Mutual Information Estimation:** For each (X_i, Y_j) pair, compute Normalized Mutual Information:
$$\mathrm{NMI}(X_i, Y_j) = \frac{I(X_i; Y_j)}{\sqrt{H(X_i) \cdot H(Y_j)}}$$
   where $I(X_i; Y_j)$ is the mutual information and H is Shannon entropy. Continuous values are discretized into histogram bins for estimation. NMI ranges from 0 (independent) to 1 (deterministic function).
4. **Leakage Detection:** For each output field Y_j, identify input fields X_i with NMI above a leakage threshold (default $\theta = 0.1$). The output field inherits the maximum classification of all leaking input fields. If no input fields leak into an output field, it receives the lowest classification (generated, not derived).
5. **Classification Assignment:** The overall transform classification is the maximum across all output field classifications.

Theoretical Foundation — Data Processing Inequality. For any Markov chain $X \to \mathrm{WASM} \to Y$:
$$I(X; Y) \leq H(X)$$
A transform can only discard information, never create new information about its inputs. Measuring the actual NMI preserved tells the system the true leakage.

Why NMI Over Simpler Approaches.

Approach	Limitation
Field name matching	Misses renames, derived fields, embedded values
Value equality checking	Misses partial leakage, bucketing, encoding
Static WASM analysis	Requires WASM decompilation expertise, misses runtime behavior
NMI over synthetic samples	Black-box method that catches renames, partial leakage, bucketing, and encoding regardless of transform internals

The key insight is that NMI measures the statistical relationship between inputs and outputs without requiring any understanding of the transform's internal logic. A transform that renames "diagnosis" to "condition" will show NMI ≈ 1.0 between those fields, correctly detecting the passthrough.

Example. A transform reads medical records with fields: `name` (LOW), `age` (LOW), `diagnosis` (HIGH), `prescription_list` (HIGH). Its outputs are: `patient_name`, `age_group`, `condition_text`.

NMI Matrix — Case A (Naive Transform)

Input ↓ / Output →	`patient_name`	`age_group`	`condition_text`
`name` **(LOW)**	0.93 LEAK	0.01	0.02
`age` **(LOW)**	0.01	0.41 LEAK	0.02
`diagnosis` **(HIGH)**	0.01	0.02	0.87 LEAK
`rx_list` **(HIGH)**	0.01	0.01	0.03

Figure 6: NMI matrix for Case A (naive transform). Red cells indicate NMI above the leakage threshold ($\theta = 0.1$), meaning the output field carries information from the corresponding input field. The `condition_text` output leaks from `diagnosis` (HIGH), so the overall transform classification is HIGH.

Case A — Naive transform (renames and passes through): Output classifications:

- `patient_name`: LOW (NMI > threshold only from `name`, which is LOW)
- `age_group`: LOW (NMI > threshold only from `age`, which is LOW)
- `condition_text`: HIGH (NMI > threshold from `diagnosis`, which is HIGH)

Overall transform classification: **HIGH** (correct — diagnosis information leaked into `condition_text`)

Case B — Improved transform (genuine downgrade): The transform is rewritten to map diagnosis to a boolean `has_chronic_condition` (true/false). Because a boolean can carry at most 1 bit of entropy while the input diagnosis field has ∼4.1 bits, the NMI is structurally limited:

- `diagnosis` → `has_chronic_condition`: NMI = 0.08 (below threshold — the boolean loses most of the diagnosis specificity)
- `prescription_list` → `has_chronic_condition`: NMI = 0.02 (independent)

Output classifications:

- `patient_name`: LOW
- `age_group`: LOW
- `has_chronic_condition`: LOW (no input field leaks above threshold)

Overall transform classification: **LOW** — a verified genuine downgrade. The system has confirmed that this transform strips sensitive medical detail rather than merely renaming it.

API Endpoints

The schema service exposes the following HTTP endpoints:

Schema Management:

- List all schema names
- Register a new schema (returns Added, AlreadyExists, or Expanded with mutation mappers)
- Get all schemas with full definitions
- Find similar schemas by name (Jaccard index on field name sets)
- Batch check schema reuse (read-only check without registration)
- Get specific schema by identity hash

Transform Management:

- Register a new transform (triggers classification analysis)
- List transforms
- Get transform metadata
- Download transform WASM bytecode
- Find similar transforms
- Verify transform hash

Claims

1. A computer-implemented method for automated schema deduplication and canonical field resolution in a distributed database system, comprising:

    a. computing a deterministic identity hash for each schema based on a human-readable descriptive name and sorted, deduplicated field names using a cryptographic hash function;

    b. detecting semantically equivalent schema names using neural embedding similarity with a configurable cosine similarity threshold;

    c. resolving incoming fields to canonical names by embedding field descriptions (excluding field names), finding nearest canonical matches above a similarity threshold, and applying bidirectional best-match verification to prevent many-to-one mappings;

    d. recording mutation mappers that track field renames from original to canonical names for downstream data transformation;

    e. returning one of three outcomes for each submitted schema: Added (new schema), AlreadyExists (identical identity hash), or Expanded (same descriptive name, superset fields).

2. The method of claim 1, wherein the bidirectional best-match verification comprises:

    a. for an incoming field F and a candidate canonical field C, verifying that C has the highest similarity to F among all canonical fields;

    b. additionally verifying that F has the highest similarity to C among all incoming fields in the current schema;

    c. accepting the mapping only if both conditions are met;

    such that no two incoming fields map to the same canonical field.

3. The method of claim 1, wherein schema expansion comprises:

    a. detecting that a new schema shares its descriptive name with an existing schema but has a different identity hash;

    b. computing a semantic field rename map between old and new schema fields using context-enriched embeddings;

    c. blocking the old schema;

    d. creating a superset schema where shared fields are mapped to the existing schema's data storage molecules via field mappers and new fields receive fresh molecules;

    e. updating the descriptive name index to point to the new schema;

    such that schema evolution preserves existing data without migration.

4. The method of claim 1, wherein field descriptions used for embedding computation are built by:

    a. preferring AI-generated field descriptions that are semantically specific to the field's meaning;

b. falling back to classification-based descriptions augmented with a schema context suffix when AI descriptions are unavailable;

c. excluding the field name from the embedding input to avoid bias from naming conventions across different data sources.

5. A computer-implemented method for information-theoretic classification of data transform outputs, comprising:

   a. resolving input field classifications from a canonical registry to establish an input ceiling (maximum classification across all input fields);

   b. generating type-appropriate synthetic test data for each input field, with N samples per field;

   c. executing the transform on synthetic data while varying each sensitive input field independently and holding other fields at baseline values;

   d. computing Normalized Mutual Information (NMI) between each input-output field pair across the N samples;

   e. identifying output fields with NMI above a leakage threshold as carrying information from the correlated input fields, inheriting their classification;

   f. assigning each output field the maximum classification of all input fields that leak into it above the threshold;

   g. assigning the overall transform classification as the maximum across all output field classifications.

6. The method of claim 5, wherein the Normalized Mutual Information is computed as:

$$\mathrm{NMI}(X, Y) = \frac{I(X; Y)}{\sqrt{H(X) \cdot H(Y)}}$$

   where $I(X; Y)$ is the mutual information estimated from discretized histograms of the input-output sample pairs, and H is Shannon entropy, and wherein NMI ranges from 0 (statistically independent) to 1 (deterministic function).

7. The method of claim 5, wherein the method detects information leakage regardless of the transform's internal implementation by measuring statistical correlation between inputs and outputs, thereby detecting field renames, partial leakage through bucketing or encoding, and information embedding without requiring static analysis of the transform code.

8. The method of claim 5, wherein Phase 1 (input ceiling) serves as a safety fallback: if Phase 2 is inconclusive (insufficient samples, WASM execution errors, or insufficient output entropy), the transform is conservatively assigned the input ceiling classification.

9. A system for automated schema management and transform classification comprising:

   a. a schema service maintaining a canonical field registry mapping field names to descriptions, types, data classifications, and interest categories;

   b. an embedding model configured to compute vector representations of schema names and field descriptions for similarity detection;

   c. a schema identity hash system providing deterministic, content-addressed schema identification;

d. a schema expansion mechanism that creates superset schemas with molecule-level data pointers preserving existing data;

e. a transform classification engine configured to execute WASM transforms on synthetic data and compute Normalized Mutual Information between input-output field pairs to determine output sensitivity classifications.

Abstract

A system and method for automated schema deduplication, canonical field resolution, and information-theoretic transform classification in distributed database systems. Schemas are identified by deterministic content hashes based on descriptive names and field names. Semantically equivalent schemas and fields are detected using neural embedding similarity with bidirectional verification, resolving naming inconsistencies across independent data sources. A global canonical field registry maintains authoritative field definitions with data classifications. Schema expansion preserves existing data through molecule-level pointers when schemas evolve with additional fields. Data transform output classifications are determined automatically using Normalized Mutual Information (NMI) analysis: the system executes transforms on synthetic data, measures the statistical relationship between each input-output field pair, and identifies information leakage regardless of the transform's internal implementation. This enables automated sensitivity downgrade verification — confirming that a transform that claims to strip sensitive fields actually does so — without manual security review.

Prepared for provisional patent filing by Edge Vector Foundation.
Filing date: [TO BE DETERMINED]
This document constitutes a provisional patent application under 35 U.S.C. 111(b).